

Architektonický návrh a implementační plán: Systém NeuroMan (v1.0)

Úvodní analýza definuje systém NeuroMan jako pokročilou vrstvu sémantického orchestrátoru, která neslouží jako monolitický jazykový model, nýbrž jako inteligentní směrovač (router) úloh v reálném čase. V současném paradigmatu vývoje umělé inteligence narážejí monolitické modely na fundamentální limity, pokud jsou nuceny spravovat diametrálně odlišné kontexty současně. Požadavek na udržení přesného abstraktního syntaktického stromu (AST) v rámci kódování, paralelně s komplexní strukturou hudebního projektu v digitální zvukové stanici (DAW), vede k degradaci pozornosti modelu (attention collapse), plýtvání tokeny a neúnosné latenci. NeuroMan tento problém eliminuje návrhem tenké, vysoce optimalizované rozhodovací vrstvy, která na základě sémantické klasifikace záměru deleguje specifické požadavky na specializované agenty a nástroje.

Tento přístup optimalizuje spotřebu tokenů, snižuje latenci a maximalizuje přesnost tím, že systém dynamicky vybírá správné "ruce" pro daný úkol. Pro programování je využit autonomní nástroj Claude Code nebo OpenCode Go, pro hudební produkci a konzultace slouží dedikovaný LLM směrovaný přes OmniRoute a pro přímou manipulaci s Ableton Live je nasazena vrstva Model Context Protocol (MCP) zprostředkovaná nástrojem Producer Pal. Následující expertní zpráva představuje detailní návrh první verze (v1.0) tohoto systému, včetně kompletního softwarového návrhu, datových toků a implementačních vzorů pro jazyk Python s ohledem na absolutní eliminaci nutnosti manuálního kopírování kontextu mezi rozhraními.

Architektonická koncepce a oddělení zodpovědností

Základní filozofie systému NeuroMan spočívá v oddělení zodpovědností (Separation of Concerns). Místo implementace masivního modelu, který by se snažil porozumět všem doménám současně, je NeuroMan navržen jako asynchronní dispečer. Tento orchestrátor propojuje tři odlišné ekosystémy do jediného unifikovaného rozhraní příkazové řádky (CLI) nebo chatu. Uživatel tak interaguje s jediným vstupním bodem, zatímco vnitřní logika autonomně volí nejvhodnější cestu k vykonání úlohy.

Tento design je postaven na špičkových lokálních a proxy technologiích, které minimalizují závislost na jediném cloudovém poskytovateli a maximalizují flexibilitu. Následující tabulka ilustruje základní rozložení komponent a jejich rolí v celkové architektuře.

Komponenta	Primární role v systému NeuroMan	Technologický základ a specifikace
OmniRoute	Jádro pro modelový routing a LLM inferenci. Zajišťuje klasifikaci záměrů a řeší	AI Gateway (Node.js/TypeScript) s lokálním OpenAI-kompatibilním API

	hudebně-teoretické dotazy.	(localhost:20128/v1), kompresi tokenů a čtyřúrovňovým fallback mechanismem ¹ .
Producer Pal	Řízení Ableton Live a manipulace s Live Object Model (LOM). Slouží jako bridge mezi AI a DAW.	Max for Live device s vestavěným Node.js serverem podporujícím jak MCP, tak REST API na portu 3350 ³ .
Claude Code / OpenCode Go	Autonomní kódovací agenti pro provádění změn v repozitáři, refaktORIZACI a analýzu softwarového kódu.	CLI aplikace volané orchestrátorem přes asynchronní subprocess, využívající neinteraktivní parametry pro bezproblémový tok dat ⁵ .
VS Code & Cline	Lokální vývojové prostředí, kde NeuroMan reálně běží. Cline slouží jako asistent přímo v editoru.	IDE s připravenými rozšířeními pro lokální agentické interakce, doplňující orchestrátora o přímou vizualizaci změn v kódu.
NeuroMan Core	Mozek systému (Python 3.10+). Analyzuje vstup, udržuje relační kontext napříč nástroji a zajišťuje CLI smyčku.	Asynchronní I/O architektura (asyncio), využívající knihovnu rich pro pokročilé formátování v terminálu.

Architektura jasně definuje, že Ableton control zůstává exkluzivně na vrstvě MCP reprezentované nástrojem Producer Pal. Neexistuje zde žádná snaha reimplementovat starší a robustní řešení jako LivePilot; namísto toho se NeuroMan soustředí na rozšiřování a volání existujících nástrojů, které jsou plně otestované a funkční. Tento modulární přístup umožňuje budoucí integraci vlastních rozšíření, aniž by bylo nutné přepisovat komunikační jádro směrem k hudebnímu softwaru.

Páteřní vrstva: OmniRoute jako inteligentní LLM brána

Pro fungování vrstvy, která musí neustále rozhodovat o směrování dotazů ("router"), je kritické mít k dispozici robustní infrastrukturu pro inferenci jazykových modelů. Tradiční přístup, kdy by NeuroMan napřímo volal API OpenAI nebo Anthropicu, by vedl k rychlému vyčerpání kvót,

vysokým finančním nákladům a neschopnosti elegantně řešit výpadky služby. Z tohoto důvodu je jako primární páteř systému nasazen OmniRoute.

OmniRoute funguje jako svobodná, open-source lokální AI brána, která sjednocuje přes 160 nezávislých poskytovatelů umělé inteligence (včetně více než 90 bezplatných tierů) pod jediný OpenAI-kompatibilní koncový bod¹. NeuroMan se tak nemusí starat o správu tokenů, vyvažování zátěže nebo obnovování spojení. Veškerá tato složitost je abstrahována.

Konfigurace strategií "Combos" pro NeuroMana

Klíčovým inovačním konceptem, který OmniRoute nabízí, jsou takzvané "combos" (kombinace). Jedná se o pojmenované řetězce modelů spojené se specifickou strategií směrování (routing strategy)⁸. Aby NeuroMan dokázal dynamicky volit mezi levným modelem pro triviální rozhodování a silným modelem pro komplexní kompozici, vyžaduje definici dvou oddělených řetězců v konfiguraci OmniRoute.

Prvním řetězcem je neuroman-fast. Tento řetězec je určen výhradně pro klasifikaci záměru (Intent Routing) v reálném čase. Pokaždé, když uživatel zadá text do CLI, NeuroMan musí ve zlomku sekundy rozhodnout, jaké nástroje aktivovat. Pro tento účel neuroman-fast využívá nejrychlejší dostupné bezplatné modely s latencí pod 500 milisekund, typicky dostupné přes poskytovatele jako Pollinations, Groq nebo Kiro, a to zcela bez nutnosti API klíčů¹. Strategie tohoto řetězce je nastavena na "Cost Optimized", což znamená, že OmniRoute automaticky směruje dotaz na nejlevnější dostupný uzel v síti⁸.

Druhým řetězcem je neuroman-strong. Ten je aktivován pouze tehdy, když klasifikátor určí, že uživatel vyžaduje hlubokou hudební analýzu, návrh kompozice, nebo potřebuje vygenerovat složitý JSON payload pro manipulaci v Ableton Live. Tento řetězec prioritizuje vysoce schopné modely, jako jsou Claude 3.5 Sonnet nebo GPT-4o. Aby se předešlo přerušení práce v důsledku vyčerpání limitů, neuroman-strong využívá inteligentní čtyřúrovňový "auto-fallback" mechanismus¹. Pokud dojde k vyčerpání předplatného (například u Claude Pro), požadavek automaticky a neviditelně "přepadne" na API klíč jiného poskytovatele, případně na levný tier, a nakonec na bezplatné modely, což zaručuje nulové prostoje během kreativního procesu⁸.

Kompresí promptů pro efektivitu tokenů

Práce s hudebními projekty generuje masivní množství dat. Když NeuroMan vyžádá stav projektu z Ableton Live (Live Set), výsledný objekt může obsahovat detailní strukturu všech stop, scén, zařízení a MIDI klipů. Odeslání takového kontextu do LLM při každém dotazu by drasticky zpomalilo inferenci a vyčerpalo kontextové okno. OmniRoute tento problém řeší integrovanou pětistupňovou kompresní pipeline (využívající techniky RTK a Caveman), která je schopna transparentně snížit objem odesílaných tokenů o 15 % až 75 %¹. NeuroMan tak může LLM modelům předávat obrovské datové struktury k analýze s jistotou, že OmniRoute na pozadí odstraní redundance, deduplikuje opakující se obsah z předchozích kol konverzace, a přitom striktně ochrání kritické bloky (jako je strukturovaný JSON obsahující notové zápisy) před kompresní distorzí⁷.

Integrace DAW: Producer Pal a flexibilita rozhraní

Pro přímé řízení Ableton Live je architektura založena na nástroji Producer Pal. Tento nástroj byl původně koncipován jako Model Context Protocol (MCP) server zabalený přímo do Max for Live zařízení, což umožňuje umělé inteligenci číst, vytvářet a upravovat Live Set pomocí přirozeného jazyka⁹. Koncept MCP slouží jako univerzální standardizovaná komunikační vrstva, která nahrazuje nutnost budovat vlastní můstky pro každý jednotlivý nástroj¹¹. Uživatel v zadání výslovně specifikoval, že správa Abletonu má probíhat přes tuto MCP vrstvu, aniž by se systém pokoušel o reimplementaci stávajících řešení.

Ačkoli Producer Pal primárně komunikuje s klienty (jako je Claude Desktop) přes protokol MCP, jeho architektura paralelně vystavuje tytéž nástroje prostřednictvím otevřeného REST API. Toto REST API běží na stejném serveru, standardně na lokálním portu 3350, a je dostupné, jakmile je Max for Live zařízení v Abletonu aktivní⁴. Pro orchestrátor napsaný v jazyce Python (NeuroMan) představuje přímé využití tohoto REST API nejelegantnější a nejrobustnější způsob, jak realizovat požadavky uživatele zadané v CLI. Integrace klasického stdio MCP klienta do asynchronní Python aplikace vyžaduje složitou manipulaci se standardními vstupy a výstupy podprocesů, zatímco REST API umožňuje bezproblémové využití standardních asynchronních HTTP knihoven (např. httpx). Z hlediska funkcionality jsou obě rozhraní naprosto identická – REST API zprostředkovává přístup ke stejným MCP nástrojům (jako ppal-read-live-set, ppal-create-clip nebo ppal-update-clip) a aplikuje shodnou validační logiku⁴.

Manipulace s Live Object Model a formátování odpovědí

Při komunikaci přes Producer Pal REST API je nezbytné správně specifikovat formát odpovědi. Standardně Producer Pal (a jeho MCP server) vrací výsledky jako kompaktní textový řetězec v syntaxi JavaScript-literálů (necitované klíče, bez bílých znaků), což je agresivně optimalizováno pro úsporu tokenů v LLM⁴. Nicméně pro Python orchestrátor, který potřebuje data dále programově zpracovávat nebo je agregovat do kontextu pro jiné agenty, je tento formát nevhodný. Proto NeuroMan ke každému volání REST API připojuje parametr `?format=json`⁴. Tento parametr donutí server vrátit plnohodnotný parsovaný JSON objekt (např. `body.result.tempo`), oddělený od případných varování z Ableton API, což radikálně zjednodušuje parsování odpovědi uvnitř souboru `ableton.py`⁴. Dále NeuroMan využívá parametr `timeoutMs`, který lze specifikovat u dlouhotrvajících operací (např. masivní hromadné generování klipů), aby se předešlo předčasnému ukončení požadavku ze strany HTTP klienta. Tento timeout může být nastaven až na 60 000 milisekund⁴.

Dalším kritickým konfiguračním krokem pro správné fungování hudební sémantiky je nastavení notace v rámci zařízení Producer Pal. Z výroby nástroj preferuje formát `bar|beat` (kompaktní, lidsky čitelný text). Pro kódovací agenty a programovou manipulaci přes LLM (jako to dělá NeuroMan) je však mnohem vhodnější strojově čitelný formát. NeuroMan proto na začátku relace odesílá konfigurační příkaz (přes `POST /config`), kterým přepne notaci na `midi-json`¹². Tento formát kóduje MIDI noty jako pole JSON objektů s vynechanými výchozími hodnotami a krátkými klíči, což dramaticky snižuje pravděpodobnost halucinací syntaxe ze strany modelů, které jsou primárně trénovány na produkci kódu a strukturovaných dat¹².

V případech, kdy specializované nástroje (`ppal-read-live-set`, `ppal-duplicate`, `ppal-select`) nestačí k provedení vysoce specifického požadavku, umožňuje Producer Pal nízkourovňový

zásah do Ableton Live Object Model (LOM) přes nástroj `ppal-live-api`¹⁰. Tento nástroj je ve výchozím stavu deaktivován pro svou potenciální destruktivitu, ale NeuroMan jej může dynamicky povolit zasláním konfigurace `{"liveApiEnabled": true}`, čímž získá absolutní programatickou kontrolu nad jakýmkoli parametrem DAW⁴.

Kódovací dispečink a autonomní agenti

Architektura NeuroMana definuje jasnou hranici mezi pochopením záměru a jeho samotným provedením. Pokud uživatel zadá požadavek na vytvoření nového softwarového modulu, úpravu existujícího kódu nebo spuštění testů, klasifikátor (`core.py`) tento záměr identifikuje jako `CODING_TASK`. V tu chvíli NeuroMan nezačne sám analyzovat lokální soubory; místo toho přebírá roli dispečera a deleguje úkol na specializované kódovací agenty, konkrétně na Claude Code nebo OpenCode Go, případně synchronizuje kontext pro VS Code rozšíření Cline. Pro delegaci na Claude Code využívá NeuroMan modul `coder.py`, který spouští CLI klienta v asynchronním podprocesu. Velkou výhodou nástroje Claude Code je podpora parametru `-p` (nebo `--print`), který přepíná aplikaci do neinteraktivního režimu⁶. V tomto režimu Claude Code přijme prompt jako argument, autonomně provede veškeré operace nad repozitářem, vypíše výsledek na standardní výstup (`stdout`) a následně se ukončí⁶. Tím je zajištěno, že interaktivní výzvy (prompty) Claude Code neblokují asynchronní smyčku NeuroMana. Zvláště užitečná je možnost spojit tento příkaz s výstupem do formátu JSON (přes další případné flagy), což umožňuje NeuroManovi strukturovaně zachytit výsledky kompilace nebo změny souborů a předložit je uživateli v jednotném rozhraní.

Alternativou pro zpracování kódovacích úloh, specifikovanou v zadání, je OpenCode Go. Jedná se o nízkonákladovou službu poskytující spolehlivý přístup k populárním open-source kódovacím modelům, často zaměřeným na čínské AI kódovací platformy, které jsou obsluhovány skrze rozhraní kompatibilní s OpenAI a Anthropic⁵. OpenCode Go lze rovněž ovládat přes příkazovou řádku, což znamená, že modul `coder.py` dynamicky sestavuje shell příkazy (např. `opencode-go run "prompt"`) na základě preferencí uživatele a spouští je stejným asynchronním mechanismem. Tím je zachována konzistence – NeuroMan odesílá požadavky, streamuje průběžný výstup (`stdout` a `stderr`) zpět do svého unifikovaného chatu s využitím barevného kódování přes knihovnu `rich` a po ukončení agenta plynule přebírá zpět řízení konverzace.

Správa kontextu a eliminace fragmentace

Jedním z nejzásadnějších problémů, které NeuroMan řeší, je roztříštěnost pracovního toku (workflow fragmentation). Tradičně musel producent a vývojář kopírovat odpovědi z rozhraní OmniRoute, vkládat je do VS Code a manuálně spravovat Ableton Live přes oddělená chybová hlášení. Návrh "jedno rozhraní pro všechno" znamená, že NeuroMan drží nadřazený relační stav. Když uživatel vznesne složitý, víceoborový požadavek (např. *"Zjisti tempo mého Ableton projektu a napiš mi Python skript, který vygeneruje MIDI klip v tomto tempu"*), NeuroMan tuto úlohu rozloží:

1. Skrze `tools/ableton.py` neviditelně získá stav projektu (včetně hodnoty tempa) voláním `ppal-read-live-set` s parametrem `?format=json`.

2. Získanou hodnotu tempa NeuroMan interně zakomponuje do textového promptu.
3. Tento obohacený prompt následně předá do tools/coder.py, který instanciuje kódovacího agenta (Claude Code).
4. Kódovací agent vytvoří požadovaný skript na disku.
5. NeuroMan agreguje výsledek a odpoví uživateli v jednom uceleném chatovacím vlákně.

Uživatel nikdy neprovádí mechanický přenos dat (copy-paste). Orchestrátor funguje jako paměťová a transportní sběrnice (message bus) mezi izolovanými agenty. To je podpořeno schopnostmi OmniRoute, který disponuje integrovaným systémem paměti (s podporou vektorů a FTS5) a dokáže napříč rotujícími klíči a modely udržovat kontinuitu konverzace pomocí strukturovaných předávacích souhrnů². Tento mechanismus zabraňuje tomu, aby NeuroMan ztratil přehled o předchozích krocích, a to i v případě, že se na pozadí vystřídá několik různých jazykových modelů.

Detailní implementační blueprint

Následující sekce předkládá hlubokou analýzu a konkrétní kódové struktury jednotlivých modulů systému NeuroMan. Implementace je navržena pro Python 3.10+ a plně využívá moderní asyncio vzory pro neblokující vstupně/výstupní operace. Kód zachovává požadovanou stromovou strukturu projektu.

Vstupní bod a uživatelské rozhraní (main.py)

Tento modul slouží jako srdce celého uživatelského zážitku. Startuje asynchronní událostní smyčku (event loop) a zachycuje vstupy uživatele. K zajištění profesionálního a přehledného výstupu se využívá balíček rich. Jádrem je nekonečná smyčka zachycující výzvy, které jsou okamžitě předávány orchestrátoru.

```
Python
# neuroman/main.py
import sys
import os
from rich.console import Console
from rich.panel import Panel
from rich import print as rprint

console = Console()

async def interactive_loop():
    """Hlavní smyčka pro interakci uživatele s rozhraním NeuroMan."""
    console.print(Panel("[bold cyan]NeuroMan v1.0 (Orchestrator Layer) [/bold cyan]\n"))
```

```

    "Všechny systémy online. Připojen k OmniRoute, VS Code a Ableton Live."
    expand=False

orchestrator = NeuroManOrchestrator(console)

while True:
    try:
        # Zachycení víceřádkového nebo komplexního promptu
        user_input = console.input("\n[bold green]NeuroMan>[/bold green] ")

        # Bezpečné ukončení vrstvy
        if user_input.strip().lower() in 'exit' 'quit' 'konec':
            console.print("[yellow]Ukončuji relaci. Agenti deaktivováni.[yellow]")
            break

        if not user_input.strip():
            continue

        # Předání požadavku jádru orchestrátoru pro sémantickou analýzu
        await orchestrator.dispatch_task(user_input)

    except KeyboardInterrupt:
        # Zachycení Ctrl+C bez pádu stack trace
        console.print("\n[yellow]Běh přerušen uživatelem (SIGINT).[yellow]")
        break

    except Exception as e:
        console.print(f"[bold red]Kritická chyba v hlavní smyčce:[/bold red] {str(e)}")

def main():
    """Entry point aplikace zajišťující správné spuštění asynchronní smyčky."""
    try:
        asyncio.run(interactive_loop())
    except RuntimeError:
        # Fallback mechanismus, pokud je smyčka již aktivní (např. v interaktivních prostředích)
        loop = asyncio.get_event_loop()
        loop.run_until_complete(interactive_loop())

if __name__ == "__main__":
    main()

```

Tento přístup odděluje prezentační logiku (CLI formátování) od analytické logiky orchestrátoru. Použití asynchronní smyčky je naprosto kritické, protože volání sítě (OmniRoute API, Producer Pal REST) nebo čekání na ukončení podprocesu kódovacího agenta by v synchronním kódu

zmrazilo celé rozhraní, což je pro moderní AI workflow nepřijatelné.

Jádro a rozhodovací matice (core.py)

Mozek systému, NeuroManOrchestrator, je instanciován z main.py a drží odkazy na všechny podřízené nástroje. Jeho úkolem není vygenerovat konečnou odpověď, ale provést tzv. dispatch (dispečink) požadavku ke správnému komponentu na základě klasifikace z router.py.

```
Python
# neuroman/core.py
import sys
from rich.console import Console
from router import OmniRouteClient
from tools.coder import dispatch_to_coder
from tools.ableton import AbletonController
from tools.web import handle_web_query

class NeuroManOrchestrator:
    """Sémantická vrstva rozhodující o směrování úloh."""

    def __init__(self, console: Console):
        self.console = console
        self.router = OmniRouteClient()
        self.ableton = AbletonController()

    async def dispatch_task(self, prompt: str):
        """Hlavní analytická metoda. Zjišťuje záměr a deleguje zpracování."""
        with self.console.status: "[cyan]Klasifikuji sémantiku dotazu (OmniRoute Fast)...[/cyan]"
        intent = await self.router.get_intent(prompt)

        self.console.print(f"[dim italic]Rozpoznání vektor úlohy: {intent}[/dim italic]")

        # Matice směrování založená na identifikovaném záměru
        if intent == "CODE":
            self.console.print("[magenta]Deleguji na kódovacího agenta (Claude Code / OpenCode Go)...[/magenta]")
            await dispatch_to_coder(prompt, self.console)

        elif intent == "ABLETON":
```



```

        self.console.print("[blue]Kontaktuji Producer Pal (MCP/REST vrstva) v Ableton Live...[/blue]")
        await self.handle_ableton_task(prompt)

    elif intent == "MUSIC":
        self.console.print("[yellow]Aktivuji model pro analýzu hudební produkce...[/yellow]")
        system_msg = "Jsi elitní hudební producent a expert na kompozici a mixáž. Odpovídej analyticky."
        # Využívá se silný model pro zajištění vysoké kvality rady
        response = await self.router.generate_completion(prompt, system_message=system_msg)
        self.console.print(f"\n[bold yellow]Hudební Expertise:[/bold yellow]\n{response}\n")

    else:
        # Kategorie GENERAL (běžné vyhledávání nebo nepřirazené úlohy)
        self.console.print("[white]Dotaz směřuje do obecné znalostní báze...[/white]")
        await handle_web_query(prompt, self.router, self.console)

    async def _handle_ableton_task(self, prompt: str):
        """
        Řeší složité propojení LLM a LOM (Live Object Model) přes Producer Pal.
        Nejprve získá kontext z DAW, předloží jej LLM pro vygenerování JSON příkazu, a ten exekuuje.
        """
        # 1. Čtení stavu projektu (Context Gathering)
        set_state = await self.ableton.read_live_set()
        if "error" in set_state:
            self.console.print(f"[red]Selhání spojení s Producer Pal: {set_state['error']}[/red]")
            return

        # 2. Příprava metapromptu pro silný model
        system_msg = f"""
        Jsi specializovaný asistent pro ovládání Ableton Live přes rozhraní Producer Pal API.
        Zde je aktuální serializovaný stav projektu (Live Set): {json.dumps(set_state)}

        Analyzuj požadavek a vygeneruj validní volání příslušného nástroje.
        K dispozici máš mj. 'ppal-create-clip', 'ppal-update-clip', 'ppal-delete', atd.
        Odpověz STRIKTNĚ jako validní JSON objekt. Žádný markdown, žádný text okolo.
        Příklad: {{"tool": "ppal-create-clip", "args": {{"trackIndex": 0, "slot": "0/0", "length": "4bar", "notes":
        "[...]"}}}}
        """

        with self.console.status("[cyan]LLM (OmniRoute Strong) generuje parametry pro Ableton...[/cyan]"):
            llm_response = await self.router.generate_completion(prompt, system_message=system_msg)

```

```

# 3. Zpracování generovaného JSON příkazu a provedení akce
try
    # Odstranění případných halucinací markdown bloků
    clean_json = llm_response.replace("`json` ", "").replace("`", "").strip()
    command_data = json.loads(clean_json)
    tool_name = command_data.get("tool")
    tool_args = command_data.get("args")

    self.console.print(f"[dim]Spouštím nástroj: {tool_name} s parametry: {tool_args}[dim]")

    # Bezprostřední volání REST API Abletonu
    result = await self.ableton.call_tool(tool_name, tool_args)
    self.console.print(f"[bold green]Exekuce v Abletonu úspěšná:[/bold green]\n{json.dumps(result, indent=2)}")

except json.JSONDecodeError:
    self.console.print("[red]Kritická chyba: LLM nevygenerovalo validní JSON strukturu.[/red]")
    self.console.print(f"[dim]Surový výstup modelu:\n{llm_response}[dim]")

except Exception as e:
    self.console.print(f"[red]Výjimka během komunikace s Abletonem: {str(e)}[/red]")

```

V této fázi je zřejmá absolutní výhoda orchestrující vrstvy. Tím, že NeuroMan sám stáhne strukturu Ableton setu skrze ppal-read-live-set⁴ a transparentně ji předhodí velkému jazykovému modelu zprostředkovanému OmniRoute, zcela eliminuje nutnost pro člověka manuálně kontrolovat stav stop a vypisovat složitou MIDI JSON syntaxi.

Komunikační můstek s modely (router.py)

Tento modul zajišťuje bezproblémové propojení s lokálně spuštěnou gatewayí OmniRoute. Využívá instanci oficiálního klienta OpenAI, avšak přepisuje base_url tak, aby směřovala na lokální proxy vrstvu.

```

Python
# neuroman/router.py
import os
from openai import AsyncOpenAI

class OmniRouteClient:

```

```
"""Klient obsluhující požadavky do 160+ LLM poskytovatelů přes OmniRoute."""
```

```
def __init__(self):
```

```
    # Přesměrování na lokální uzel OmniRoute zajišťující 4-tier fallback [cite: 1]
```

```
    self.base_url = os.getenv("OMNIRoute_URL" "http://localhost:20128/v1"
```

```
    self.api_key = os.getenv("OMNIRoute_KEY" "omniroute-local-bypass"
```

```
    self.client = AsyncOpenAI
```

```
        base_url=self.base_url
```

```
        api_key=self.api_key
```

```
    async def get_intent(self, prompt: str) -> str:
```

```
        """
```

```
        Klasifikátor záměru využívající rychlé a levné modely (např. Llama 3 8B)
```

```
        pro okamžitou odpověď s minimální latencí.
```

```
        """
```

```
        system_prompt =
```

```
            "Jsi binární klasifikátor sítě NeuroMan. Tvůj výstup musí být "
```

```
            "exkluzivně jedno slovo z množiny: [CODE, ABLETON, MUSIC, GENERAL]. "
```

```
            "CODE = programování, repozitáře, VS Code, skripty. "
```

```
            "ABLETON = ovládání hudebního softwaru, manipulace s tracky, efekty a DAW. "
```

```
            "MUSIC = hudební teorie, rady ohledně mixáže bez požadavku na akci. "
```

```
            "GENERAL = všechny ostatní informační dotazy."
```

```
        try
```

```
            response = await self.client.chat.completions.create
```

```
                model="combo-fast" # Zkonfigurováno v OmniRoute jako rychlý řetězec
```

```
                messages=
```

```
                    ("role" "system" "content" system_prompt)
```

```
                    ("role" "user" "content" prompt)
```

```
                temperature=0.0 # Striktně deterministický výstup pro klasifikaci
```

```
                max_tokens=10
```

```
            return response.choices[0].message.content.strip().upper()
```

```
        except Exception:
```

```
            # Silný fallback v případě neočekávaného kolapsu klasifikátoru
```

```
            return "GENERAL"
```

```
    async def generate_completion(self, prompt: str, system_message: str = "", model: str =  
"combo-strong") -> str:
```

```

"""
Univerzální generátor pro komplexní úlohy vyžadující silné usuzování.
Využívá schopnosti OmniRoute komprimovat prompt před odesláním (RTK/Caveman).
"""
messages = []

if system_message:
    messages.append("role" "system" "content" system_message)
    messages.append("role" "user" "content" prompt)

response = await self.client.chat_completions.create(
    model=model, # Preferuje silné modely (Claude 3.5, GPT-4o) s auto-fallbackem
    messages=messages,
    temperature=0.7
)

return response.choices[0].message.content.strip()

```

Klíčové je zde volání specifických řetězců (`model="combo-fast"` a `model="combo-strong"`). Uživatel si v grafickém rozhraní OmniRoute Dashboard dopředu nadefinuje složení těchto řetězců. NeuroMan tak do procesu výběru modelu fyzicky nezasahuje, pouze využívá robustní logiku, kterou již OmniRoute na vrstvě sítě zajišťuje⁸.

Přímé propojení s Ableton Live (`tools/ableton.py`)

Zatímco Claude Desktop s Abletonem komunikuje zdlouhavým procesem spouštění `node.js` procesu a udržování `stdio` linky podle pravidel MCP⁹, NeuroMan přistupuje k problému pragmaticky. Protože Producer Pal na svém interním Node pro Max serveru vystavuje i standardní REST API, Python klient může přistupovat ke všem funkcím přímo přes HTTP požadavky na stejném lokálním portu (3350)⁴. Tento přístup garantuje typovou bezpečnost, snadné zachytávání chyb a umožňuje NeuroManovi předformátovat požadavky do JSONu a nativně vyhodnocovat vrácená varování⁴.

```

Python
# neuroman/tools/ableton.py
import httpx

class AbletonController:
    """Wrapper pro Producer Pal REST API suplující MCP komunikační vrstvu."""

    def __init__(self, base_url="http://localhost:3350"):

```

```
self.base_url = base_url
```

```
# 60 sekundový timeout pro masivní operace s klipy a scénami
```

```
self.client = httpx.AsyncClient(timeout=60.0)
```

```
# Bezprostřední konfigurace notace pro minimalizaci LLM chyb [cite: 12]
```

```
self._ensure_config()
```

```
def _ensure_config(self):
```

```
    """
```

```
    Asynchronní optimalizace zařízení. Přepíná lidsky čitelnou notaci (bar|beat)
    na strojově orientovaný formát (midi-json), což dramaticky usnadňuje agentům
    generování přesných notových struktur.
```

```
    """
```

```
    try
```

```
        import request
```

```
        requests.post
```

```
        f"{self.base_url}/config"
```

```
        json= "notation" "midi-json"
```

```
        timeout=2.0
```

```
    except Exception:
```

```
        # Nevyhazuje výjimku, zachytí se až při reálném call_tool
```

```
        pass
```

```
async def read_live_set(self) -> dict:
```

```
    """
```

```
    Vrátí agregovanou strukturu celého Ableton projektu voláním 'ppal-read-live-set'.
```

```
    """
```

```
    return await self.call_tool("ppal-read-live-set")
```

```
async def call_tool(self, tool_name: str, args: dict) -> dict:
```

```
    """
```

```
    Abstraktní volání jakéhokoliv Producer Pal nástroje.
```

```
    Zásadní je parametr ?format=json, který garantuje strukturovanou odpověď
    (body.result), a nekompaktní string, a vyextrahování pole 'warnings'.
```

```
    """
```

```
    # Připojení format=json nutí server rozložit strukturu pro programovou kontrolu
```

```
    url = f"{self.base_url}/api/tools/{tool_name}?format=json"
```

```
    try
```

```
        response = await self.client.post
```

```
        url
```

```
        json=args
```

```

        headers={"Content-Type": "application/json"}

    if response.status_code == 200:
        data = response.json()
        # Zpracování případné logické chyby reportované Live Object Modelem
        if data.get("isError") == False:
            raise Exception(data.get("result") "Neznámá LOM výjimka z Producer Pal")

        # Zpřístupnění dat a případných varování (např. varování o kvantizaci na audio klipu)
        result_data = data.get("result")
        warnings = data.get("warnings")
        if warnings:
            # Logování varování, ale neblokování procesu
            pass
        return result_data
    elif response.status_code == 404:
        return "error" f"Nástroj '{tool_name}' nebyl nalezen nebo je deaktivován."
    else:
        return "error" f"HTTP {response.status_code}: {response.text}"

except httpx.RequestError as exc:
    return "error" f"Selhání sítě na {self.base_url}. Ujistěte se, že Producer Pal v Abletonu běží.
    Detail: {exc}"

```

Tento modul je navržen tak, aby nevyžadoval hluboké zásahy v případě, že se v Producer Pal objeví nový nástroj. Díky asertivnímu volání `call_tool` s dynamickým jménem a dynamickými argumenty dokáže LLM zavolat `ppal-update-clip`, `ppal-duplicate` nebo prohledat lokální adresáře (`ppal-search`) pomocí stejné metody s využitím standardizované JSON dokumentace, kterou server poskytuje¹⁰.

Dispečink kódovacích úloh (tools/coder.py)

Když dojde k požadavku na programování, refaktORIZACI souborů v projektu VS Code nebo vytvoření algoritmů, NeuroMan odklání tok informací směrem ke kódovacím agentům, protože se sám (jako orchestrátor) odmítá zabývat komplikovanou správou souborového systému. Uživatel specifikoval Claude Code a OpenCode Go jako cílové instance, s připraveností kooperovat i s lokálním rozšířením Cline⁶.

Při použití Claude Code CLI se využívá jeho nedávno přidaná funkce neinteraktivního provádění (non-interactive mode), kterou aktivuje přepínač `--print` nebo zkráceně `-p`⁶. Tento flag zamezuje Claude Code vykreslovat grafické prvky do terminálu a spoléhat na lidský vstup; naopak nutí agenta analyzovat prompt, vygenerovat řešení, provést ho a výstup nasměrovat do běžné UNIX roury (pipe), odkud ho asynchronní proces NeuroMana dokáže čistě přechytit.

Python

```
# neuroman/tools/coder.py
```

```
import asyncio
```

```
from rich.console import Console
```

```
async def dispatch_to_coder(prompt: str, console: Console, agent_type: str = "claude"):
```

```
    """
```

```
    Subprocess delegátor. Spustí dedikovaného agenta v izolovaném shellu a přebírá  
    jeho výstupy (stdout/stderr) zpět do chatovacího rozhraní NeuroMana.
```

```
    """
```

```
    # Detekce a příprava správného shell příkazu
```

```
    if agent_type == "claude":
```

```
        # Claude Code s využitím neinteraktivního flagu -p (automatizované skriptování)
```

```
        command = f'claude -p "{prompt}"'
```

```
    elif agent_type == "opencode":
```

```
        # OpenCode Go pro spuštění asijských/levnějších modelů pro kód
```

```
        command = f'opencode-go run "{prompt}"'
```

```
    else:
```

```
        # Fallback na jakýkoli jiný nástroj s MCP podporou
```

```
        command = f'echo "Neznámý agent: {agent_type}"'
```

```
    console.print(f"[dim]Inicializuji autonomního agenta: {command}[/dim]")
```

```
    # Asynchronní vyvolání shellu zajistí, že NeuroMan nezamrzne
```

```
    process = await asyncio.create_subprocess_shell
```

```
        command
```

```
        stdout=asyncio.subprocess.PIPE
```

```
        stderr=asyncio.subprocess.PIPE
```

```
    
```

```
    # Korutina pro kontinuální čtení výstupu po řádcích a vykreslování v reálném čase
```

```
    async def stream_output(stream, prefix, color):
```

```
        while True:
```

```
            line = await stream.readline()
```

```
            if not line:
```

```
                break
```

```
            console.print(f"[{color}][[{prefix}]][/color]] {line.decode('utf-8').rstrip()}")
```



```

# Spuštění obou streamovacích úloh paralelně
await asyncio.gather(
    stream_output(process.stdout, "AGENT" "cyan"),
    stream_output(process.stderr, "ERROR" "red"),
)

await process.wait()
console.print(f"[dim italic]Agent dokončil úlohu (Exit Code: {process.returncode})[/dim italic]"

```

V případě asistence pro VS Code (například nástroj Cline) může NeuroMan působit jako předzpracovatel (preprocessor). Místo toho, aby sám zasahoval do souborů, vygeneruje podrobné systémové instrukce a uloží je do souboru context.md, na který je Cline následně upozorněn, čímž vzniká asynchronní sémantická vazba bez kopírování¹³.

Legacy záložní modul (tools/web.py)

Pro komplexnost je zachována i logika předchozí iterace NeuroMana pro úlohy netýkající se programování ani hudby.

```

Python
# neuroman/tools/web.py
from rich.console import Console

async def handle_web_query(prompt: str, router, console: Console):
    """Fallback modul pro všeobecné dotazy, využívá OmniRoute přímo."""
    with console.status("[cyan]Vyhledávám odpověď v obecné paměti modelů...[/cyan]"):
        response = await router.generate_completion(prompt)

    console.print(f"\n[bold magenta]NeuroMan:[/bold magenta]\n{response}\n"

```

Bezpečnost, latence a spolehlivost

Navržený systém zavádí bezpečnostní a stabilizační mechanismy na třech úrovních. Na úrovni orchestrátoru eliminuje NeuroMan riziko poškození kontextu tím, že nesnaží držet v paměti stovky tisíc tokenů najednou; namísto toho dynamicky žádá o kontext pouze před aktivací příslušné větve. Na úrovni směrování zajišťuje vrstva OmniRoute nejen ochranu před výpadky pomocí mechanismů Circuit Breaker (okamžitě detekuje chyby 429/500 od poskytovatelů a přepíná uzly)¹, ale také fyzickou ochranu dat, protože všechny API klíče a logy promptů jsou

drženy lokálně na počítači vývojáře a šifrovány algoritmem AES-256-GCM, bez nutnosti odesílat citlivé kódy třetím stranám k synchronizaci². Na konec, na úrovni DAW rozhraní, poskytuje Producer Pal oddělený kontextový prostor (per-project memory), do kterého ukládá poznatky izolovaně, což zamezuje zkřížení informací mezi rozdílnými skladbami¹³.

Závěr a doporučení pro další rozvoj

Zpráva demonstruje, že transformace systému NeuroMan z monolitického dotazovače na inteligentního orchestrátora typu hub-and-spoke drasticky zvyšuje jeho efektivitu, přesnost a uživatelský komfort. Tím, že NeuroMan přijme filosofii delegace – směřující triviální rozhodování na nejrychlejší model, těžké algoritmické úlohy na specializované CLI agenty typu Claude Code a vizuálně orientovanou hudební produkci na vyhrazenou vrstvu Producer Pal – splňuje požadavek vytvořit jednotné rozhraní bez nutnosti vkládání a vyjímání kontextu mezi separátními okny⁴. Implementovaný Python návrh představuje čistý, vysoce výkonný základ, na kterém lze v budoucnosti stavět další moduly (například integrace generátorů zvuku nebo analýza logů v produkci), zatímco pevná infrastruktura OmniRoute na pozadí zajišťuje maximální úsporu nákladů a neustálou dostupnost umělé inteligence.

Citovaná díla

1. OmniRoute - Open Source AI Gateway Router - EveryDev.ai, <https://www.everydev.ai/tools/omniroute>
2. Omniroute - MseeP.ai, <https://mseeP.ai/app/diegosouzapw-omniroute>
3. Producer Pal - MCP server running inside Ableton Live for AI music production - Reddit, https://www.reddit.com/r/mcp/comments/1oz0uka/producer_pal_mcp_server_running_inside_ableton/
4. Ableton Live REST API | Producer Pal, <https://producer-pal.org/guide/rest-api>
5. OpenCode Go - Docker Docs, <https://docs.docker.com/ai/docker-agent/providers/opencode-go/>
6. Claude Code CLI Documentation: Commands, Flags, and Governance - Gravitee, <https://www.gravitee.io/blog/claude-code-cli-documentation-commands-flags-and-governance>
7. OmniRoute: The Free, Open-Source AI Gateway That Stops You From Ever Hitting a Rate Limit Again | by Fazal, <https://fazal-sec.medium.com/omniroute-the-free-open-source-ai-gateway-that-stops-you-from-ever-hitting-a-rate-limit-again-fc19fe8c65c5>
8. OmniRoute — open-source AI gateway that pools ALL your accounts, routes to 60+ providers, 13 combo strategies, 11 provi : r/opencodeCLI - Reddit, https://www.reddit.com/r/opencodeCLI/comments/1shzxyw/omniroute_opensource_ai_gateway_that_pools_all/
9. Producer Pal — Ableton MCP for AI music production, <https://producer-pal.org/>
10. Features | Producer Pal — Ableton MCP for AI music production, <https://producer-pal.org/features>
11. Ableton Live MCP Server: An AI Engineer's Deep Dive - Skywork,

- <https://skywork.ai/skypage/en/ableton-live-ai-engineer/1980466938061115392>
12. Ableton Live Agent Skill - Producer Pal, <https://producer-pal.org/guide/skills>
 13. Releases · adamjmurray/producer-pal - GitHub, <https://github.com/adamjmurray/producer-pal/releases>
 14. OpenCode Go Review: Is the \$10 AI Coding Plan Worth It? | Thomas Wiegold Blog, <https://thomas-wiegold.com/blog/opencode-go-review/>
 15. Opencode Go or Commandcode Go? - Reddit, https://www.reddit.com/r/opencode/comments/1uxslgb/opencode_go_or_commandcode_go/